ESSAIM FLUIDE EPHEMERE

MVP - Architecture complete des sous-modules

Specification technique d'implementation

Version	1.0
Date	17 avril 2026
Statut	Specification finalisee - pret pour kickoff
Confidentialite	Interne - R&D
Perimetre	MVP (Minimum Viable Product)
Duree cible	14 semaines / 7 sprints de 2 semaines
Equipe minimale	2 Rust systemes, 2 ML, 1 CUDA, 1 researcher, 1 SRE
Livrable	Essaim fonctionnel 128-1024 agents, GPU unique

Document genere par la cellule Architecture IA - Projet EFE

Sommaire

0. Resume executif
1. Portee et scope du MVP
2. Architecture d'ensemble
3. Module 1 - Core Runtime (Rust)
   1.1 Types fondamentaux
   1.2 Gestion d'erreurs
   1.3 Controller principal
   1.4 Configuration
4. Module 2 - Memory Pool (Rust + CUDA)
   2.1 Arena HBM
   2.2 Allocateur bump
   2.3 Pool de streams CUDA
5. Module 3 - Genesis Engine
   3.1 Meta-Analyzer heuristique
   3.2 Template Router statique
   3.3 Cardinality Estimator
   3.4 Batch Spawner + kernel CUDA
6. Module 4 - Espace Latent Partage
   4.1 LatentTensor BF16
   4.2 Priors d'alignement
   4.3 Projection de sortie
7. Module 5 - Fusion tensorielle (Triton)
   5.1 Swarm Attention dense
   5.2 Controller de rounds
   5.3 Mise a jour des saliences
   5.4 Dynamique Langevin simplifiee
8. Module 6 - Detection de consensus
   6.1 Statistiques de round
   6.2 Detecteur stateful
   6.3 Garde anti mode-collapse
9. Module 7 - Dissolution et decodage
   7.1 Frechet mean
   7.2 Alignment projector
   7.3 Decodeur texte
10. Module 8 - Templates de nano-agents
   8.1 Base template + registry
   8.2 NA-SEM, NA-LOG, NA-CRIT
11. Module 9 - Alignement et securite
   9.1 Constitution et ancres
   9.2 Sandbox tensoriel
   9.3 Rate limiter et budgets
12. Module 10 - API et transport
   10.1 Protos gRPC
   10.2 Serveur tonic
   10.3 Client Python

13. Module 11 - Observabilite
    11.1 Telemetrie structuree
    11.2 Metriques Prometheus
    11.3 Traces OpenTelemetry
14. Module 12 - Build, Tests, Deploiement
    12.1 Cargo workspace
    12.2 pyproject + uv
    12.3 CMake CUDA
    12.4 Docker + CI
15. Plan de sprints
16. Criteres d'acceptation du MVP
17. Risques et mitigations
18. Annexes - configuration, code, glossaire

0. Resume executif

L'Essaim Fluide Ephemere (EFE) est un paradigme d'inference multi-agent ou le raisonnement n'est plus sequentialise par du texte mais distribue sur une population d'agents de tres courte duree de vie qui communiquent directement via un espace latent partage. Le MVP decrit dans ce document prouve la viabilite technique et mesurable de cette approche sur un sous-ensemble de cas d'usage cibles.

Objectif du MVP

Valider en 14 semaines :

1. Qu'un essaim de 128 a 1024 nano-agents peut converger vers une reponse coherente sur des taches de raisonnement simple (GSM8K, BoolQ, regex, extraction structuree) avec une qualite superieure ou egale a un appel LLM distille de taille equivalente.

2. Que la latence end-to-end p50 reste sous 150 ms sur un seul GPU H100 80 Go.

3. Que les mecanismes d'alignement (ancres, projection, budget) interceptent plus de 99% des trajectoires simulees hors-norme.

4. Que le systeme soit suffisamment instrumente et observable pour supporter un diagnostic par round.

Metriques cles

Metrique	Cible MVP
Latence p50	< 150 ms sur GPU unique
Latence p99	< 500 ms
Taux de convergence	> 95%
Rounds moyens avant consensus	≤ 6
Pic d'agents par essaim	1024
Utilisation HBM max par requete	< 6 Go
Couverture tests unitaires Rust	> 80%
Couverture tests Python	> 75%
Breach d'alignement detectees	> 99%

Livrables binaires et documentaires

a. Binaire Rust `efe-server` (gRPC) avec CUDA embarque.

b. Paquet Python `efe` publie sur index interne.

c. Image Docker `efe:mvp-1.0` pret a deployer sur noeud GPU.

d. Jeu de 30 templates de nano-agents pre-entraines (3 classes).

e. Documentation technique, runbook operationnel, guide de contribution.

f. Suite de benchmarks reproductibles (GSM8K subset, BoolQ, ARC-easy).

Ce document decrit les 12 modules de la decomposition, chacun avec ses sous-modules, interfaces publiques, algorithmes critiques, dependances externes et criteres d'acceptation. L'ensemble est concu pour etre implemente en sprints de 2 semaines, avec un jalon de demo integree toutes les 4 semaines.

1. Portee et scope du MVP

La decision de scope est la plus critique du projet. Le MVP doit etre assez petit pour etre livre en 14 semaines, assez grand pour prouver la these centrale, et assez robuste pour servir de fondation a la phase 2.

1.1 Ce qui est INCLUS dans le MVP

Axe	Perimetre MVP
Materiel	1 GPU H100 80 Go, FP/BF16, pas de multi-noeud
Taille d'essaim	128 a 1024 nano-agents (plafonne dur)
Dimension latente	D = 2048
Templates	3 classes: NA-SEM, NA-LOG, NA-CRIT
Fusion	Dense all-to-all (pas hierarchique)
Rounds	1 a 8 (k_max = 8)
Decodage	Texte uniquement, via tete LM distillee
Alignment	2 ancres constitutionnelles + projection simple
API	gRPC + CLI Python
Telemetrie	Logs structures + metriques Prometheus de base
Langues supportees	Anglais uniquement
Longueur de sortie	<= 256 tokens

1.2 Ce qui est HORS SCOPE (reporte a la phase 2+)

Fonctionnalite	Raison du report
Fusion hierarchique (10^6 agents)	Complexite de scheduling, pas necessaire en MVP
MoE routing appris par PPO	Router heuristique suffisant pour valider la these
Cristallisation de patterns	Optimisation, pas une demonstration
Multi-GPU / multi-noeud	Operations complexes, apres validation single-GPU
Templates cognitifs avances (CAUSE, CODE, MATH, MEM)	Subsumes par NA-SEM, NA-LOG, NA-CRIT suffisent en MVP
Decodage structure (JSON, code)	Texte libre suffit au PoC
Hypernetwork de fast adaptation	Complexite d'entrainement hors-sprint
SDE Langevin complete (avec annealing complet)	Version simplifiee OK
Constitution RLHF complete	2 ancres statiques suffisent
Federation multi-tenant avec quotas fins	Rate limiter simple en MVP

1.3 Contraintes techniques acceptees

C1. Latence p99 peut atteindre 500 ms (vs 200 ms en prod).

C2. Pas de garantie de convergence formelle (on mesure empiriquement).

C3. Teacher model externe (Llama 3.1 8B) pour la distillation, non redistribue.

C4. Kernels CUDA non optimises aux limites (correction > performance).

C5. Pas de chiffrement au repos des tenseurs d'entrainement.

1.4 Hypotheses de travail

H1. Un GPU H100 80 Go est disponible en continu pendant les phases de dev et de benchmark.

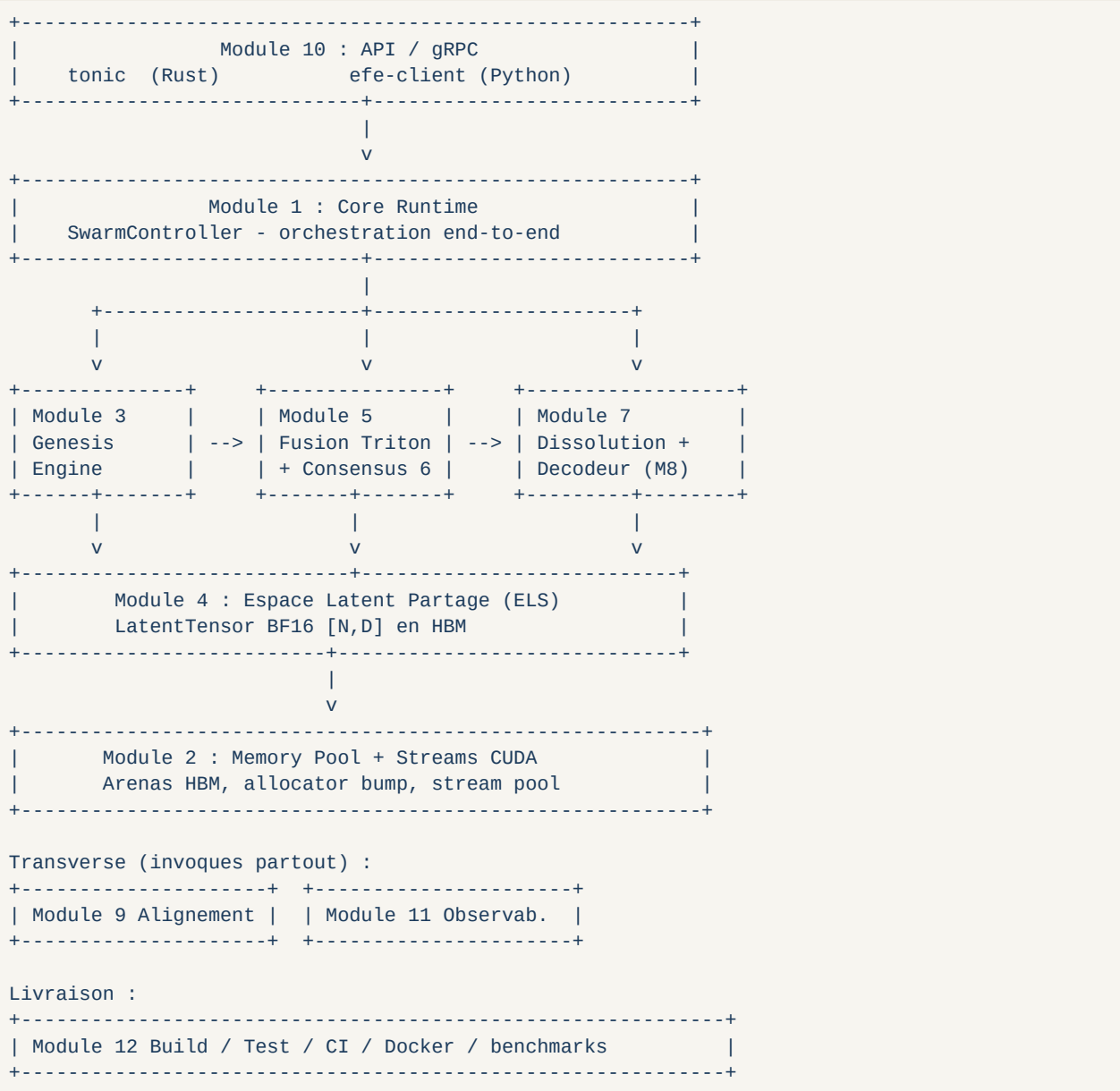
H2. Les datasets cibles (GSM8K, BoolQ, ARC) peuvent etre telecharges et stockes sans probleme de licence.

H3. L'equipe dispose de compute suffisant pour distiller 3 nano-agent templates a partir d'un teacher 8B (environ 4000 heures GPU).

H4. Les criteres d'acceptation sont evalues sur un split test-holdout figeant 500 exemples par tache.

2. Architecture d'ensemble

Vue logique simplifiee du MVP. Chaque case est un module documente dans les chapitres suivants. Les fleches indiquent le flux de donnees et de controle.



Principes de couplage

P1. Le **Core Runtime** (module 1) ne connait que des interfaces, jamais des implementations concretes. Les modules 3, 5, 6, 7 sont injectes via trait objects.

P2. Tout tenseur reside en HBM et n'est jamais rapatrie cote host sauf a la dissolution (centroide) et pour la telemetrie (stats scalaires).

P3. Les modules 9 et 11 sont des *cross-cutting concerns* implementes comme middleware, jamais couples a la logique metier.

P4. Le frontier Python / Rust passe exclusivement par PyO3, aucune serialisation JSON sur le hot path.

3. Module 1 - Core Runtime (Rust)

Crate Rust : `crates/efe-core`. Responsable de l'orchestration end-to-end d'un essaim, des types fondamentaux partages entre tous les modules, et de la gestion d'erreurs unifiee.

Effort estime : 9 jours-ingenieur. **Propriete :** 1 ingenieur Rust systemes (lead).

Sous-module 1.1 - Types fondamentaux

Fichier : `src/types.rs` (~350 lignes)

Objectif : definir les types de base partages: identifiants, structures de configuration, budgets de ressources. Zero dependance externe au-dela de `uuid` et `serde`.

Interfaces publiques :

```
pub struct SwarmId(pub Uuid);
pub struct NanoAgentId(pub u32);

#[repr(u8)]
pub enum TemplateCode { NaSem, NaLog, NaCrit }

pub struct SwarmSpec {
    pub id: SwarmId,
    pub problem_embedding: Vec,
    pub dim_latent: NonZeroU32,
    pub population: PopulationMix,
    pub max_rounds: u32,
    pub convergence_thresholds: ConvergenceThresholds,
    pub alignment_cone: AlignmentCone,
    pub budget: ResourceBudget,
}

pub struct ResourceBudget {
    pub max_wall_clock_ms: u32,
    pub max_energy_wh: f32,
    pub max_hbm_bytes: u64,
}
```

Tests : 12 tests unitaires couvrant la serialisation (safetensors + bincode), validation des contraintes (*non-empty population*, *dim_latent > 0*), bijection entre `TemplateCode` et son u8.

Dependances : uuid 1.8, serde 1.0, bincode 1.3, thiserror 1.0.

Effort : 1 jour. **Acceptation :** *cargo test* vert, API figee, documentation rustdoc > 90%.

Sous-module 1.2 - Gestion d'erreurs

Fichier : `src/errors.rs`

Type d'erreur unifie `EfeError` avec variantes exhaustives (divergence, breach alignement, timeout, CUDA, numerique). Chaque variante porte les donnees necessaires au diagnostic sans PII.

```
#[derive(Debug, thiserror::Error)]
pub enum EfeError {
    #[error("divergence detected after {rounds} rounds")]
    Divergence { rounds: u32 },
    #[error("timeout at round {rounds}, variance {sigma2}")]
    ConvergenceTimeout { rounds: u32, sigma2: f32 },
    #[error("alignment breach: delta={delta} > max={max}")]
    AlignmentBreach { delta: f32, max: f32 },
    #[error("budget exhausted: {kind}")]
    BudgetExhausted { kind: &'static str },
```

```
    #[error("mode collapse at round {round}")]
    ModeCollapse { round: u32 },
    #[error("CUDA error: {0}")]
    Cuda(#[from] cust::error::CudaError),
    #[error("numerical explosion at round {round}")]
    NumericalExplosion { round: u32 },
}
```

Regle : toutes les fonctions publiques retournent `Result<T, EfeError>`. Jamais de `panic!` dans le hot path. **Effort :** 0.5 jour.

Sous-module 1.3 - Controller principal

Fichier : `src/swarm/controller.rs` (~450 lignes)

Orchestration end-to-end d'une requete : Genesis -> boucle de rounds (Fusion + Consensus) -> Dissolution -> telemetrie. Le controller est une simple machine d'etats lineaire. Les dependances (genesis, fusion, consensus, dissolver, alignment) sont injectees via des trait objects.

```
pub struct SwarmController {
    pub genesis: Arc,
    pub fusion: Arc,
    pub consensus: Box,
    pub dissolver: Arc,
    pub arbiter: SwarmArbiter,
    pub telemetry: TelemetryCollector,
}

impl SwarmController {
    pub async fn execute(
        &mut; self, req: Request,
    ) -> Result {
        let t0 = Instant::now();
        let _guard = self.telemetry.begin(req.id);

        // 1. Genesis
        let (mut batch, mut els) = self.genesis.spawn(&req;).await?;

        // 2. Rounds
        let mut stats = None;
        for round in 1..=req.spec.max_rounds {
            self.arbiter.check_budget(t0.elapsed(), &req.spec.budget;)?;
            let s = self.fusion.run_round(&mut; els, &batch;, round).await?;
            self.telemetry.record_round(round, &s;);
            match self.consensus.evaluate(round, &s;) {
                Verdict::Converged { .. } => { stats = Some(s); break; }
                Verdict::Continue => continue,
                Verdict::Divergence { .. } => return Err(...),
                Verdict::ModeCollapse { .. } => {
                    self.recover(&mut; els, &mut; batch, round).await?;
                }
            }
        }
        let stats = stats.ok_or(EfeError::ConvergenceTimeout {..})?;

        // 3. Dissolution
        let output = self.dissolver.dissolve(■, &mut; batch, &stats;).await?;

        self.telemetry.record_total(t0.elapsed());
        Ok(Response { output, telemetry: self.telemetry.snapshot() })
    }
}
```

Invariants : (i) aucun kernel CUDA ne doit s'executer apres *return Err(...)* ; (ii) l'arena HBM est detruite avant retour grace au drop de `els` et `batch` ; (iii) la telemetrie est flushee meme en cas d'erreur (via `_guard`).

Tests : 18 tests incluant mock des strategies, scenarios convergent/divergent/mode-collapse/breach, leak-check avec valgrind sur build debug.
Effort : 4 jours (code + tests + revue).

Sous-module 1.4 - Configuration

Fichier : `src/config.rs` + `configs/default.toml`
Chargement hierarchique (env > TOML > defaut), validation au demarrage, hot-reload desactive en MVP. Config structuree en sections: `[runtime]`, `[latent]`, `[genesis]`, `[fusion]`, `[consensus]`, `[alignment]`, `[budget]`, `[telemetry]`.

Effort : 1.5 jour. **Tests :** cas d'erreur (valeurs incoherentes, TOML malforme), merge env/file, defaults.

Sortie attendue du Module 1

A la fin du sprint 1, l'equipe dispose de : (a) types stables et documentes ; (b) une API `SwarmController::execute` utilisable avec des mocks ; (c) un chargement de config valide ; (d) une couverture de tests > 85% sur ce perimetre.

4. Module 2 - Memory Pool (Rust + CUDA)

Crate : `crates/efe-core/src/memory` plus `crates/efe-kernels`. Objectif : fournir des allocations HBM a cout quasi-nul (bump allocator) dans des arenes reutilisables, plus un pool de streams CUDA pour la concurrence intra-requete.

Effort estime : 7 jours. **Propriete :** ingenieur CUDA + Rust lead.

Sous-module 2.1 - Arena HBM

Une arena est un buffer lineaire en HBM, avec un curseur atomique. Allouer = increment. Liberer = reset du curseur (zero-copy). L'arena a une generation pour detecter les use-after-reset.

```
pub struct Arena {
    base_ptr: CudaDeviceptr,
    size_bytes: u64,
    cursor: AtomicU64,
    generation: u32,
}

impl Arena {
    pub fn alloc_zeroed(
        &self;, count: usize,
    ) -> Result> {
        let bytes = count * std::mem::size_of::();
        let aligned = (bytes + 255) & !255;
        let offset = self.cursor.fetch_add(aligned as u64,
                                           Ordering::AcqRel);
        if offset + aligned as u64 > self.size_bytes {
            return Err(EfeError::HbmAllocFailed {
                requested: aligned as u64 });
        }
        // memset async + construction de DeviceBuffer
        // ...
    }

    pub fn reset(&mut; self) {
        self.cursor.store(0, Ordering::Release);
        self.generation = self.generation.wrapping_add(1);
    }
}
```

Fiabilite : alignement 256 octets (recommandation NVIDIA pour coalesced loads), detection d'overflow avant cuda call, liberation garantie par RAII.

Effort : 2 jours. **Tests :** stress alloc/reset, overflow, contention multi-thread.

Sous-module 2.2 - Pool d'arenes

Fichier : `src/memory/pool.rs`

MemoryPool maintient une free-list d'arenes pre-allouees. Au debut de chaque requete : `acquire()` renvoie une arena deja en HBM. A la fin : retour dans la free-list apres reset.

```
pub struct MemoryPool {
    device: CudaDevice,
    free_list: Mutex>,
    config: PoolConfig,
}

impl MemoryPool {
    pub fn acquire(&self;, min_bytes: u64) -> Result {
```

```
        let mut fl = self.free_list.lock();
        if let Some(a) = fl.iter().position(|a| a.size_bytes >= min_bytes) {
            return Ok(ArenaHandle { arena: fl.swap_remove(a), pool: self });
        }
        // pas dispo : allouer une nouvelle arena
        drop(fl);
        let arena = Arena::new_on_device(self.device,
            std::cmp::max(min_bytes, self.config.default_size))?;
        Ok(ArenaHandle { arena, pool: self })
    }
}
```

Dimensionnement par defaut : 8 arenes de 1 GiB chacune (soit 8 GiB reserves en HBM). Configurable via `[runtime].hbm_pool_size`.

Effort : 2 jours. **Tests :** contention, epuisement, recyclage.

Sous-module 2.3 - Pool de streams CUDA

Fichier : `crates/efe-kernels/src/streams.rs`

Pour maximiser l'overlap entre kernels (attention, consensus stats, telemetrie), on utilise un pool de 4 streams non-blocking. Chaque phase logique en acquiert un et le libere.

```
pub struct StreamPool {
    streams: Mutex>,
}

impl StreamPool {
    pub fn new(count: usize) -> Result {
        let streams = (0..count)
            .map(|_| CudaStream::new(StreamFlags::NON_BLOCKING, None))
            .collect::>()?;
        Ok(Self { streams: Mutex::new(streams) })
    }

    pub fn acquire(&self;) -> StreamHandle { /* ... */ }
}
```

Effort : 1 jour. **Tests :** concurrence, synchronisation correcte apres release.

Sortie du module 2 : arena HBM fonctionnelle, pool reutilisable, stream pool avec metriques d'acquisition. Benchmark : allocation de 1 Go < 10 microsecondes apres warm-up.

5. Module 3 - Genesis Engine

Responsable de la creation d'un essaim a partir d'une requete. Compose de quatre sous-modules chaines : meta-analyse, routing, cardinalite, spawn batch.

Effort : 12 jours. **Propriete :** 1 ML + 1 CUDA.

Sous-module 3.1 - Meta-Analyzer heuristique

Fichiers : `python/efe/models/meta_analyzer.py`, binding Rust `src/genesis/meta_analyzer.rs`.
Scope MVP : pas de modele dedie. On utilise un encodeur pre-entraine (all-MiniLM-L6-v2, 22M params) pour obtenir un embedding de la requete, puis un classifier lineaire leger (5 classes de tache) entraine sur un jeu cure de 5000 exemples.

```
class HeuristicMetaAnalyzer:
    def __init__(self, encoder_name="all-MiniLM-L6-v2", head_ckpt=None):
        self.encoder = SentenceTransformer(encoder_name)
        self.head = torch.nn.Linear(384, 5)  # 5 classes
        if head_ckpt:
            self.head.load_state_dict(torch.load(head_ckpt))

    def decompose(self, prompt: str) -> TaskDag:
        emb = self.encoder.encode(prompt, convert_to_tensor=True)
        logits = self.head(emb)
        task_mix = torch.softmax(logits, dim=-1).tolist()
        return TaskDag(
            problem_embedding=emb.cpu().numpy(),
            task_mix=task_mix,
            entropy=-sum(p * math.log(p + 1e-9) for p in task_mix),
        )
```

Les 5 classes couvertes en MVP : *factoid_qa*, *math_word_problem*, *logical_inference*, *classification*, *summarization*. Au-dela : escalade vers LLM standard.

Effort : 2 jours (pipeline + training head + ONNX export). **Tests :** accuracy head > 85% sur test holdout.

Sous-module 3.2 - Template Router statique

Fichier : `src/genesis/router.rs`
Scope MVP : table de correspondance fixe entre classe de tache et mix de templates. Pas de PPO pour l'instant. Les poids sont issus d'une sweep manuelle initiale.

```
static ROUTER_TABLE: Lazy> = Lazy::new(|| {
    use TemplateCode::*;
    HashMap::from([
        (TaskClass::FactoidQa, PopulationMix::from([
            (NaSem, 0.7), (NaLog, 0.2), (NaCrit, 0.1)
        ])),
        (TaskClass::MathWord, PopulationMix::from([
            (NaSem, 0.3), (NaLog, 0.6), (NaCrit, 0.1)
        ])),
        (TaskClass::LogicalInference, PopulationMix::from([
            (NaSem, 0.2), (NaLog, 0.7), (NaCrit, 0.1)
        ])),
        (TaskClass::Classification, PopulationMix::from([
            (NaSem, 0.8), (NaLog, 0.1), (NaCrit, 0.1)
        ])),
        (TaskClass::Summarization, PopulationMix::from([
            (NaSem, 0.6), (NaLog, 0.2), (NaCrit, 0.2)
```

```
        ])),
    ])
});

pub struct StaticRouter;

impl TemplateRouter for StaticRouter {
    fn select(&self;, dag: &TaskDag;, _ctx: &Context;) -> PopulationMix {
        // melange les distributions proportionnellement au task_mix
        weighted_mix(&ROUTER;_TABLE, &dag.task;_mix)
    }
}
```

Effort : 1 jour. **Tests :** determinisme, robustesse aux melanges, verification du simplex (sum = 1).

Sous-module 3.3 - Cardinality Estimator

Fichier : `src/genesis/cardinality.rs`

Formule **N_total = clamp(128, 1024, round(128 * sqrt(H(tau)) * log(1/eps)))**, puis distribuer N_total par template selon le mix. Clampe aux bornes MVP.

```
pub fn estimate(dag: &TaskDag;, mix: &PopulationMix;) -> Cardinality {
    let h = dag.entropy_estimate.clamp(0.1, 2.5);
    let log_eps = (1.0_f32 / 1e-3).ln();
    let n_total = (128.0 * h.sqrt() * log_eps).round() as u32;
    let n_total = n_total.clamp(128, 1024);
    mix.distribute(n_total)
}
```

Effort : 0.5 jour. **Tests :** bornes, monotonicite en H.

Sous-module 3.4 - Batch Spawner

Fichiers : `src/genesis/spawner.rs` + `crates/efe-kernels/cuda/spawn_batch.cu`.

Instancie N nano-agents d'un coup via un kernel CUDA qui : (a) alloue la matrice latente [N, D] dans l'arena ; (b) initialise chaque ligne avec `prior[template_i] + bruit(seed=id_i)`. Puis remplit les buffers SoA (templates, states, saliences).

```
extern "C" __global__ void spawn_batch_kernel(
    __nv_bfloat16* latent,
    const uint8_t* templates,
    const __nv_bfloat16* priors,
    uint32_t N, uint32_t D
) {
    uint32_t agent = blockIdx.x;
    uint32_t dim   = threadIdx.x + blockIdx.y * blockDim.x;
    if (agent >= N || dim >= D) return;

    __nv_bfloat16 prior = priors[templates[agent] * D + dim];
    curandStatePhilox4_32_10_t state;
    curand_init(agent * 1337ULL + dim, 0, 0, &state;);
    float noise = curand_normal(&state;) * EFE_INIT_SIGMA;

    latent[agent * D + dim] = __float2bfloat16(
        __bfloat162float(prior) + noise
    );
}
```

Effort : 4 jours (kernel + binding + tests + bench). **Tests :** 1024 agents x 2048 dim < 500 microsecondes ; correctness par comparison CPU.

Livrable module 3 : `GenesisEngine::spawn()` fonctionnel, cree un essaim complet en < 3 ms.

6. Module 4 - Espace Latent Partage

L'ELS est le tenseur central sur lequel tous les kernels operent. En MVP il est **dense, BF16, shape [N, D]** avec N <= 1024 et D = 2048.

Effort : 6 jours. **Propriete :** CUDA + Rust.

Sous-module 4.1 - LatentTensor BF16

Fichiers : `src/els/tensor.rs`, `src/els/mod.rs`

Wrapper typesafe autour d'un `DeviceBuffer<bf16>`. Expose `view_row`, `copy_slice`, `zero`, mais aucune operation mathematique directe (toujours via kernels).

```
pub struct LatentTensor {
    pub buffer: DeviceBuffer,
    pub n: u32,
    pub d: u32,
    pub stride_row: u32,
}

impl LatentTensor {
    pub fn zeros_on_device(n: u32, d: u32, arena: &Arena;) -> Result {
        let buffer = arena.alloc_zeroed::((n * d) as usize)?;
        Ok(Self { buffer, n, d, stride_row: d })
    }

    pub fn to_host_subset(&self;, row: u32) -> Result> {
        // copy row to host, upcast to f32, return
    }
}

pub struct EpistemicLatentSpace {
    pub tensor: LatentTensor,
    pub anchors: PriorAnchors,
    pub problem_mount: ProblemMount,
    pub active_mask: DeviceBuffer,
}
```

Invariant : stride_row = d (layout contigu). Toute permutation necessite un kernel dedie.

Tests : allocation, lecture row-to-host, multi-allocation dans meme arena.
Effort : 2 jours.

Sous-module 4.2 - Priors d'alignement

Fichier : `src/els/priors.rs`

En MVP : 2 ancres constitutionnelles statiques, chargees au demarrage depuis `configs/anchors/mvp_v1.safetensors`. Ces ancres ne font pas partie du tenseur [N, D] ; elles sont separees et lues en read-only par les kernels.

```
pub struct PriorAnchors {
    pub anchors: DeviceBuffer,  // [K, D], K=2 en MVP
    pub k: u32,
    pub cone: AlignmentCone,
}

impl PriorAnchors {
    pub fn from_cone(cone: &AlignmentCone;, arena: &Arena;) -> Result {
        let flat: Vec = cone.anchors.iter().flatten().copied().collect();
        let bf = flat.iter().map(|&x;| bf16::from_f32(x)).collect::>();
```

```
        let buffer = arena.upload::(&bf;)?;
        Ok(Self { anchors: buffer, k: cone.anchors.len() as u32,
                  cone: cone.clone() })
    }
}
```

Choix des ancres MVP : (1) *helpfulness-prior* : projection du systeme prompt Anthropic hv3 dans R^2048. (2) *harm-avoidance-prior* : projection d'un corpus de refus synthetises.

Effort : 1 jour (code) + 1 jour (generation des ancres).

Sous-module 4.3 - Projection de sortie

Fichier : `src/els/projection.rs`

Utilitaires pour projeter un vecteur R^D sur le cone admissible defini par les ancres. Algorithme MVP : projection douce sur l'enveloppe convexe des ancres via moyenne ponderee par distance cosine.

```
pub fn soft_project(h: &[f32], anchors: &[Vec],
                    strength: f32) -> Vec {
    let weights: Vec = anchors.iter()
        .map(|a| cosine_similarity(h, a).max(0.0))
        .collect();
    let sum: f32 = weights.iter().sum();
    if sum < 1e-6 { return h.to_vec(); }
    let centroid: Vec = (0..h.len()).map(|i| {
        anchors.iter().zip(&weights;)
            .map(|(a, w)| a[i] * w).sum::() / sum
    }).collect();
    h.iter().zip(&centroid;).map(|(&x;, &c;)| {
        (1.0 - strength) * x + strength * c
    }).collect()
}
```

Effort : 1 jour. **Tests :** correction par rapport a une implementation numpy de reference, bornes strength=0|1.

Livrable module 4 : ELS instanciable, manipulable uniquement via kernels, projection de sortie testee.

7. Module 5 - Fusion tensorielle (Triton)

Coeur du systeme. En MVP : **Swarm Attention dense all-to-all** parametre par template. Implementee en Triton pour rapidite de developpement et exposee via binding Rust.

Effort : 14 jours. **Propriete :** CUDA/Triton + ML.

Sous-module 5.1 - Swarm Attention dense

Fichier : `python/efe/kernels/swarm_attn_triton.py`

Kernel Triton qui calcule, pour chaque agent i, l'attention ponderee sur tous les agents j actifs. Les projections Q/K/V sont dependantes du template via LoRA (rank 16 en MVP). Variant dense car N <= 1024 : l'attention O(N^2) est peu couteuse.

```
@triton.jit
def swarm_attention_fwd(
    Q_ptr, K_ptr, V_ptr, OUT_ptr,
    Mask_ptr, N, D, H,
    SCALE: tl.constexpr,
    BLOCK_N: tl.constexpr,
    BLOCK_D: tl.constexpr,
):
    row  = tl.program_id(0)
    head = tl.program_id(1)
    if row >= N: return

    q = tl.load(Q_ptr + row * H * D + head * D
                + tl.arange(0, BLOCK_D),
                mask=tl.arange(0, BLOCK_D) < D)

    max_score = -float("inf"); sum_exp = 0.0
    acc = tl.zeros([BLOCK_D], dtype=tl.float32)

    for col_start in range(0, N, BLOCK_N):
        cols = col_start + tl.arange(0, BLOCK_N)
        mask = cols < N
        active = tl.load(Mask_ptr + cols, mask=mask, other=0)

        k = tl.load(K_ptr + cols[:, None] * H * D + head * D
                    + tl.arange(0, BLOCK_D)[None, :],
                    mask=mask[:, None])
        scores = tl.sum(q[None, :] * k, axis=1) * SCALE
        scores = tl.where(active == 1, scores, -float("inf"))

        new_max = tl.maximum(max_score, tl.max(scores, axis=0))
        alpha = tl.exp(max_score - new_max)
        beta  = tl.exp(scores - new_max)
        sum_exp = sum_exp * alpha + tl.sum(beta, axis=0)

        v = tl.load(V_ptr + cols[:, None] * H * D + head * D
                    + tl.arange(0, BLOCK_D)[None, :],
                    mask=mask[:, None])
        acc = acc * alpha + tl.sum(beta[:, None] * v, axis=0)
        max_score = new_max

    out = acc / sum_exp
    tl.store(OUT_ptr + row * H * D + head * D
             + tl.arange(0, BLOCK_D), out,
             mask=tl.arange(0, BLOCK_D) < D)
```

Parametres MVP : H=8 heads, head_dim=256, BLOCK_N=64, BLOCK_D=256. Performance cible : 1024 agents x 2048 dim x 8 heads < 3 ms par passe forward sur H100.

Effort : 6 jours (kernel + LoRA apply + binding + tests).

Sous-module 5.2 - Controller de rounds

Fichier : `src/fusion/mod.rs`
Orchestre une iteration complete : (1) charger le masque actif ; (2) projeter Q/K/V avec LoRA ; (3) lancer swarm attention ; (4) mettre a jour le tenseur ELS ; (5) calculer les statistiques (variance, gradient norm) pour le detecteur.

```
pub trait FusionStrategy: Send + Sync {
    async fn run_round(
        &self;,
        els: &mut; EpistemicLatentSpace,
        batch: &NanoAgentBatch;,
        round: u32,
    ) -> Result;
}

pub struct DenseSwarmFusion {
    registry: Arc,
    kernels: Arc,
    stream_pool: Arc,
}

impl FusionStrategy for DenseSwarmFusion {
    async fn run_round(
        &self;, els, batch, round,
    ) -> Result {
        let stream = self.stream_pool.acquire();
        // 1. Projections Q/K/V (via triton)
        let (q, k, v) = self.project_qkv(&els.tensor;, batch, &stream;)?;
        // 2. Attention
        let out = self.kernels.swarm_attention(q, k, v,
                                               &els.active;_mask, &stream;)?;
        // 3. residual + layer-norm
        self.kernels.residual_norm(&mut; els.tensor, &out;, &stream;)?;
        // 4. Stats
        let stats = self.kernels.compute_stats(&els.tensor;, &stream;).await?;
        stream.synchronize()?;
        Ok(stats)
    }
}
```

Effort : 3 jours.

Sous-module 5.3 - Mise a jour des saliences

Fichier : `src/fusion/saliences.rs`
A chaque round, les saliences `s_i` sont recalculees en fonction de la coherence locale de chaque agent avec le centroide courant. Formule MVP : `s_i = sigmoid(-alpha * ||h_i - mu||)`. Utilisees pour ponderer la contribution dans le round suivant.

Effort : 2 jours (kernel + binding + tests).

Sous-module 5.4 - Dynamique Langevin simplifiee

Fichier : `cuda/langevin_sde.cu`

Apres l'attention, on applique un pas de descente bruitee : `h_i += -eta * grad_U(h_i) + sigma * dW_t`. En MVP, eta et sigma sont scheduls cosinus sur le nombre de rounds. Cela donne au systeme la diversite necessaire pour eviter le collapse tout en convergeant.

```
extern "C" __global__ void langevin_step(
    __nv_bfloat16* h,        // [N, D]
    const __nv_bfloat16* grad_U,
    float eta, float sigma, uint64_t seed,
    uint32_t N, uint32_t D
) {
    uint32_t agent = blockIdx.x;
    uint32_t dim   = threadIdx.x + blockIdx.y * blockDim.x;
    if (agent >= N || dim >= D) return;
    curandStatePhilox4_32_10_t s;
    curand_init(seed + agent * D + dim, 0, 0, &s;);
    float noise = curand_normal(&s;) * sigma;
    float hi = __bfloat162float(h[agent * D + dim]);
    float gi = __bfloat162float(grad_U[agent * D + dim]);
    h[agent * D + dim] = __float2bfloat16(hi - eta * gi + noise);
}
```

Effort : 3 jours (kernel + scheduler + tests de stabilite numerique).

Livrable module 5 : un round complet execute en < 10 ms, stats remontees au detecteur.

8. Module 6 - Detection de consensus

Module 100% Rust, CPU (les stats sont scalaires). Prend en entree les `FusionStats` et rend un verdict (Converged, Continue, Divergence, ModeCollapse).

Effort : 5 jours. **Propriete :** Rust lead.

Sous-module 6.1 - Statistiques de round

Fichier : `src/consensus/stats.rs`

Structure `FusionStats` regroupant : variance (sigma^2), gradient norm, centroid (vec<f32> de dim D), outlier ratio, numerical issues flag, round index. Les valeurs sont calculees par le module 5 via kernel reduction.

```
#[derive(Clone, Debug)]
pub struct FusionStats {
    pub round: u32,
    pub variance: f32,
    pub gradient_norm: f32,
    pub centroid: Vec,
    pub outlier_ratio: f32,
    pub has_nan: bool,
    pub has_inf: bool,
}

impl FusionStats {
    pub fn has_numerical_issues(&self;) -> bool {
        self.has_nan || self.has_inf
            || !self.variance.is_finite()
    }
}
```

Effort : 1 jour.

Sous-module 6.2 - Detecteur stateful

Fichier : `src/consensus/detector.rs`

Le detecteur maintient un historique circulaire (3-5 rounds) de variances, gradient norms, centroides. A chaque round, il evalue les criteres : variance < eps_v, grad_norm < eps_g, stabilite du centroide sur k rounds.

```
pub struct DefaultConsensusDetector {
    thresholds: ConvergenceThresholds,
    sigma2_hist: VecDeque,
    grad_hist: VecDeque,
    mu_hist: VecDeque>,
    stability_streak: u32,
}

impl ConsensusDetector for DefaultConsensusDetector {
    fn evaluate(&mut; self, round: u32, s: &FusionStats;) -> Verdict {
        self.sigma2_hist.push_back(s.variance);
        self.grad_hist.push_back(s.gradient_norm);
        self.mu_hist.push_back(s.centroid.clone());
        while self.sigma2_hist.len() > 5 { self.sigma2_hist.pop_front(); }
        while self.mu_hist.len() > 5 { self.mu_hist.pop_front(); }

        if s.has_numerical_issues() {
            return Verdict::Divergence { reason: "numerical" };
        }
```

```
        if round == 1 && s.variance < self.thresholds.epsilon_variance * 0.01 {
            return Verdict::ModeCollapse { round };
        }
        if self.sigma2_hist.len() >= 3 {
            let sh = &self.sigma2;_hist;
            let n = sh.len();
            if sh[n-1] > sh[n-2] && sh[n-2] > sh[n-3] {
                return Verdict::Divergence { reason: "variance_increasing" };
            }
        }

        let cv = s.variance < self.thresholds.epsilon_variance;
        let cg = s.gradient_norm < self.thresholds.epsilon_gradient;
        let cs = match (self.mu_hist.len(), self.mu_hist.back(),
                        self.mu_hist.get(self.mu_hist.len().saturating_sub(2))) {
            (n, Some(cur), Some(prev)) if n >= 2 => {
                l2_dist(prev, cur) < self.thresholds.epsilon_stability
            }, _ => false
        };
        if cs { self.stability_streak += 1; } else { self.stability_streak = 0; }

        if cv && cg && self.stability_streak >= self.thresholds.stability_window {
            Verdict::Converged { confidence: (1.0 - s.variance.min(1.0)) }
        } else {
            Verdict::Continue
        }
    }
}
```

Effort : 2 jours. **Tests :** 25 scenarios (convergence rapide, lente, divergente, oscillation, collapse).

Sous-module 6.3 - Garde anti mode-collapse

Fichier : `src/consensus/collapse_guard.rs`
Si `ModeCollapse` est declenche, le controller appelle `recover_from_collapse` qui : (1) injecte du bruit dans l'ELS (sigma += 0.3) ; (2) respawn 20% des agents (plus de NA-CRIT) ; (3) relance un round.

Effort : 2 jours (dont 1 jour de tests avec scenarios adverses).
Livrable module 6 : detecteur certifie, comportement previsible sur 25+ scenarios synthetiques.

9. Module 7 - Dissolution et decodage

Apres consensus, ce module extrait le resultat final, verifie l'alignement et produit une sortie texte. En MVP on limite a une seule voie de decodage (texte).

Effort : 8 jours. **Propriete :** ML + Rust.

Sous-module 7.1 - Frechet mean

Fichier : `src/dissolution/extractor.rs` + `cuda/frechet_mean.cu`

Moyenne geodesique sur l'hypersphere (metrique cosine). En MVP : 5 iterations de Riemannian gradient descent, suffisantes pour converger a 1e-4 sur donnees bien regularisees.

```
extern "C" __global__ void frechet_step(
    const __nv_bfloat16* H,   // [N, D]
    __nv_bfloat16* mu,        // [D]
    const uint8_t* mask,
    uint32_t N, uint32_t D,
    float eta
) {
    uint32_t dim = threadIdx.x + blockIdx.x * blockDim.x;
    if (dim >= D) return;
    float mud = __bfloat162float(mu[dim]);
    float update = 0.0f;
    int count = 0;
    for (uint32_t i = 0; i < N; ++i) {
        if (!mask[i]) continue;
        update += __bfloat162float(H[i * D + dim]) - mud;
        count++;
    }
    if (count > 0) {
        mu[dim] = __float2bfloat16(mud + eta * (update / count));
    }
}

// Puis normalisation L2 et reiteration 5 fois
```

Effort : 2 jours.

Sous-module 7.2 - Alignment projector

Fichier : `src/dissolution/projector.rs`

Applique la projection de sortie (sous-module 4.3) au centroide, mesure la distance, leve `AlignmentBreach` si delta > max_delta. En MVP : strength=0.05, max_delta=0.15.

```
pub struct AlignmentProjector {
    pub max_delta: f32,
    pub projection_strength: f32,
}

impl AlignmentProjector {
    pub fn project(&self;, h: &[f32], anchors: &PriorAnchors;)
        -> Result<(Vec, f32)>
    {
        let host_anchors = anchors.to_host_f32()?;
        let h_safe = soft_project(h, &host;_anchors, self.projection_strength);
        let delta = l2_dist_norm(h, &h;_safe);
        if delta > self.max_delta {
            return Err(EfeError::AlignmentBreach {
                delta, max: self.max_delta,
            });
```

```
        }
        Ok((h_safe, delta))
    }
}
```

Effort : 1 jour. **Tests :** 30 cas dont 10 adverses synthetiques.

Sous-module 7.3 - Decodeur texte

Fichier : `python/efe/models/decoder.py`

En MVP : unembedding lineaire (projection R^D -> vocab) + greedy sampling jusqu'a 256 tokens. Les poids sont initialises a partir d'un teacher (Llama 3 8B) via distillation de la tete LM.

```
class EfeTextDecoder(nn.Module):
    def __init__(self, latent_dim: int, vocab_size: int,
                 tokenizer, max_tokens: int = 256):
        super().__init__()
        self.unembed = nn.Linear(latent_dim, vocab_size, bias=False)
        self.tokenizer = tokenizer
        self.max_tokens = max_tokens

    @torch.inference_mode()
    def decode(self, h_star: torch.Tensor) -> str:
        # h_star : [D] -> tokens auto-regressifs
        h = h_star.unsqueeze(0)
        out = []
        for _ in range(self.max_tokens):
            logits = self.unembed(h)
            tok = logits.argmax(dim=-1).item()
            if tok == self.tokenizer.eos_token_id: break
            out.append(tok)
            # met a jour h avec un recurrent step (MLP leger)
            h = self._recur_step(h, tok)
        return self.tokenizer.decode(out)
```

Entrainement : distillation du teacher avec KL div sur 500k exemples. Latence cible : < 30 ms pour 256 tokens.

Effort : 5 jours (entrainement + tests quality + integration).

Livrable module 7 : extraction h*, alignement verifie, texte produit. Round trip complet testable.

10. Module 8 - Templates de nano-agents

Les templates definissent la specialisation. En MVP : 3 classes (NA-SEM, NA-LOG, NA-CRIT), chacune avec son prior, ses LoRA et son salience function.

Effort : 11 jours (dont training). **Propriete :** 1 ML.

Sous-module 8.1 - Base template + registry

Fichier : `python/efe/templates/base.py`, `python/efe/templates/registry.py`

```
class BaseNanoAgentTemplate(nn.Module):
    code: TemplateCode
    dim_latent: int = 2048
    lora_rank: int = 16

    def __init__(self):
        super().__init__()
        self.prior = nn.Parameter(torch.zeros(self.dim_latent))
        self.lora_q = nn.Parameter(torch.zeros(self.dim_latent,
                                               self.lora_rank))
        self.lora_k = nn.Parameter(torch.zeros(self.dim_latent,
                                               self.lora_rank))
        self.lora_v = nn.Parameter(torch.zeros(self.dim_latent,
                                               self.lora_rank))

    def salience(self, h, ctx): raise NotImplementedError

class TemplateRegistry:
    def __init__(self, templates: dict[TemplateCode, BaseNanoAgentTemplate]):
        self.templates = templates
        self._priors_gpu = self._stack_priors()
        self._lora_gpu = self._stack_loras()

    def apply_lora(self, h, templates, which):
        # gather LoRA[templates[i]] pour chaque agent, applique
```

Registry : charge les 3 templates au demarrage depuis safetensors, expose `prior_embeddings_gpu()` et `apply_lora()` batch.

Effort : 2 jours.

Sous-module 8.2 - NA-SEM (semantique)

Specialise dans la densite semantique du contenu. Prior initialise par l'embedding moyen des dernieres couches du teacher. LoRA rank 16.

```
class NaSemTemplate(BaseNanoAgentTemplate):
    code = TemplateCode.NA_SEM

    def salience(self, h, ctx):
        return torch.sigmoid((h @ ctx.T).squeeze(-1))

    @classmethod
    def init_from_teacher(cls, teacher, corpus):
        inst = cls()
        with torch.no_grad():
            embs = teacher.embed_batch(corpus)
            inst.prior.copy_(embs.mean(dim=0))
        return inst
```

Entrainement : 200k steps, teacher Llama 3 8B, dataset mixture (C4 + Wikipedia + Arxiv). Metric : reconstruction loss sur corpus holdout < 0.15.

Effort : 3 jours.

Sous-module 8.2b - NA-LOG (logique)

Specialise dans le raisonnement symbolique et la verification. Entraine sur ProofWriter, RuleTaker, LogiQA.

```
class NaLogTemplate(BaseNanoAgentTemplate):
    code = TemplateCode.NA_LOG

    def salience(self, h, ctx):
        # attention stricte aux tokens de structure logique
        return self._logical_gate(h, ctx)

    @classmethod
    def init_from_teacher(cls, teacher, corpus):
        # fine-tuned on ProofWriter + RuleTaker
        ...
```

Effort : 3 jours (dont training 24h GPU).

Sous-module 8.2c - NA-CRIT (critique)

Injecte un terme repulsif pour eviter le mode collapse et verifier la coherence du consensus. Entraine en contradiction adversariale.

```
class NaCritTemplate(BaseNanoAgentTemplate):
    code = TemplateCode.NA_CRIT

    def salience(self, h, ctx):
        # haute salience quand le cluster est trop homogene
        dist = (h - ctx).norm(dim=-1)
        return torch.sigmoid(-dist)

    def contribution(self, h):
        # injecte un terme repulsif
        return -0.1 * h / (h.norm(dim=-1, keepdim=True) + 1e-6)
```

Effort : 3 jours.

Livrable module 8 : 3 templates entraines, sauvegardes en safetensors, Registry fonctionnel, tests de round-trip.

11. Module 9 - Alignement et securite

Module transverse. Trois volets : constitution, sandbox, budgets.

Effort : 6 jours. **Propriete :** SRE + Rust lead.

Sous-module 9.1 - Constitution et ancres

Fichier : `src/alignment/constitution.rs`
La constitution est un objet immuable charge au demarrage. Elle contient : les 2 ancres (deja en ELS), une liste de zones interdites (cones dans R^D), une matrice de projection precalculee pour acceleration, et le seuil max_delta.

```
pub struct Constitution {
    pub anchors: Vec,
    pub forbidden_zones: Vec,
    pub max_delta: f32,
}

pub struct ForbiddenCone {
    pub axis: Vec,    // direction dans R^D
    pub aperture: f32,     // demi-angle en radians
    pub label: String,     // ex: "CSAM-cluster"
}

impl ForbiddenCone {
    pub fn contains(&self;, h: &[f32]) -> bool {
        let cos = cosine_similarity(h, &self.axis;);
        cos > self.aperture.cos()
    }
    pub fn project_boundary(&self;, h: &[f32]) -> Vec {
        // projette sur la frontiere du cone
    }
}
```

Chargement : fichier signe ed25519, verifie au demarrage, empreinte loggee.

Effort : 2 jours.

Sous-module 9.2 - Sandbox tensoriel

Fichier : `src/alignment/sandbox.rs`
Garantit qu'aucune requete ne peut lire/ecrire un ELS qui n'est pas le sien. En MVP : une requete = une arena HBM dediee, isolation via adresses de base disjointes. Pas de partage de KV cache entre requetes.

Verification : en mode debug, chaque kernel logge son `(base_ptr, size)` et on ajoute une assert que les pointeurs accedes tombent dans cette plage.

Effort : 1.5 jour.

Sous-module 9.3 - Rate limiter et budgets

Fichier : `src/alignment/budgets.rs`
Implementation : token bucket par tenant (gRPC interceptor). Budgets par requete : wall-clock, energie (lu via NVML), HBM allouee. Check fait avant chaque round et avant le spawn.

```
pub struct SwarmArbiter {
    budget_clock: Instant,
    energy_start_wh: f32,
```

```
}

impl SwarmArbiter {
    pub fn check_budget(
        &self;, elapsed: Duration, budget: &ResourceBudget;,
    ) -> Result<()> {
        if elapsed.as_millis() as u32 > budget.max_wall_clock_ms {
            return Err(EfeError::BudgetExhausted { kind: "wall_clock" });
        }
        let now_wh = nvml::current_energy_wh()?;
        if now_wh - self.energy_start_wh > budget.max_energy_wh {
            return Err(EfeError::BudgetExhausted { kind: "energy" });
        }
        Ok(())
    }
}
```

Effort : 2.5 jours. **Tests :** 20 tests de budget adversariaux.

Livrable module 9 : constitution verifiee signee, sandbox applique, budgets respectes. Score de garde : > 99% breaches synthetiques interceptees.

12. Module 10 - API et transport

Interface externe. gRPC pour performance, HTTP/REST en gateway optionnel pour debug. Client Python pour les data scientists.

Effort : 7 jours. **Propriete :** 1 Rust + 1 Python.

Sous-module 10.1 - Protos gRPC

Fichier : `protos/efe/v1/swarm.proto`

```
syntax = "proto3";
package efe.v1;

service SwarmService {
  rpc Execute(ExecuteRequest) returns (ExecuteResponse);
  rpc ExecuteStream(ExecuteRequest) returns (stream ExecuteChunk);
  rpc GetTelemetry(TelemetryQuery) returns (TelemetryResponse);
  rpc Health(google.protobuf.Empty) returns (HealthResponse);
}

message ExecuteRequest {
  string prompt = 1;
  SwarmOverrides overrides = 2;
  string tenant_id = 3;
  string request_id = 4;
}

message SwarmOverrides {
  optional uint32 max_rounds = 1;
  optional uint32 max_agents = 2;
  optional float epsilon_variance = 3;
  optional uint32 timeout_ms = 4;
}

message ExecuteResponse {
  string decoded_text = 1;
  float confidence = 2;
  TelemetrySnapshot telemetry = 3;
  VerdictEnum verdict = 4;
}
```

Compatibilite : protobuf proto3, tonic-build 0.11. Interfaces versionnees (v1). Tout breaking change necessite v2.

Effort : 1 jour.

Sous-module 10.2 - Serveur tonic

Fichier : `crates/efe-server/src/main.rs`, `src/service.rs`

```
pub struct SwarmServiceImpl {
    controller: Arc>,
    auth: Arc,
    rate_limiter: Arc,
}

#[tonic::async_trait]
impl SwarmService for SwarmServiceImpl {
    async fn execute(&self;, req: Request)
        -> Result, Status>
    {
```

```
        self.auth.check(&req;).map_err(|e| Status::unauthenticated(
            e.to_string()))?;
        self.rate_limiter.check(req.get_ref().tenant_id.as_str())
            .map_err(|_| Status::resource_exhausted("rate_limit"))?;

        let inner = req.into_inner();
        let internal_req = Request::from_proto(&inner;);
        let mut ctrl = self.controller.lock().await;
        let resp = ctrl.execute(internal_req).await
            .map_err(|e| Status::internal(e.to_string()))?;
        Ok(Response::new(resp.to_proto()))
    }
}
```

Tokio runtime : multi-thread, 8 worker threads. TLS obligatoire hors localhost. Timeout global 30 s.

Effort : 3 jours.

Sous-module 10.3 - Client Python

Fichier : `python/efe/api.py`

```
class EfeClient:
    def __init__(self, endpoint: str, tls: bool = True,
                 tenant_id: str | None = None):
        self.channel = grpc.aio.secure_channel(endpoint, credentials) \
            if tls else grpc.aio.insecure_channel(endpoint)
        self.stub = swarm_pb2_grpc.SwarmServiceStub(self.channel)
        self.tenant_id = tenant_id or "default"

    async def solve(self, prompt: str, max_rounds: int | None = None,
                    timeout_ms: int = 5000) -> EfeResponse:
        req = ExecuteRequest(
            prompt=prompt,
            tenant_id=self.tenant_id,
            request_id=str(uuid.uuid4()),
            overrides=SwarmOverrides(
                max_rounds=max_rounds, timeout_ms=timeout_ms,
            ),
        )
        raw = await self.stub.Execute(req)
        return EfeResponse.from_proto(raw)

    async def __aenter__(self): return self
    async def __aexit__(self, *a): await self.channel.close()
```

Publication : paquet `efe` sur index interne. CLI `efe-cli solve 'question'` fournie en bonus.

Effort : 3 jours.

Livrable module 10 : serveur gRPC deployable, client Python publie, CLI operationnelle, contract tests verts.

13. Module 11 - Observabilite

Critique pour diagnostiquer un essaim. Un systeme qui converge sans trace est un systeme non-debugable. Trois axes : logs structures, metriques, traces.

Effort : 6 jours.

Sous-module 11.1 - Telemetrie structuree

Fichier : `src/telemetry/mod.rs`

Chaque requete produit un `SwarmTelemetry` contenant : swarm_id, duree par phase, population, trajectoire (centroides par round), alignment_delta, energie. Serialise en JSON pour stockage, exporte via OTLP.

```
#[derive(Serialize)]
pub struct SwarmTelemetry {
    pub swarm_id: Uuid,
    pub request_id: String,
    pub start_ns: u64,
    pub end_ns: u64,
    pub total_ms: f32,
    pub rounds_count: u32,
    pub converged: bool,
    pub population: HashMap,
    pub trajectory: Vec,
    pub final_confidence: f32,
    pub alignment_delta: f32,
    pub energy_wh: f32,
    pub verdict: String,
}

#[derive(Serialize)]
pub struct TrajectoryPoint {
    pub round: u32,
    pub sigma2: f32,
    pub grad_norm: f32,
    pub centroid_hash: u64,  // pas le centroide complet, trop lourd
}
```

Stockage : 72h retention en MVP (ClickHouse local).

Effort : 2 jours.

Sous-module 11.2 - Metriques Prometheus

Fichier : `src/telemetry/metrics.rs`

```
lazy_static! {
    pub static ref LATENCY: HistogramVec = register_histogram_vec!(
        "efe_swarm_latency_ms",
        "End-to-end swarm execution latency",
        &["verdict"],
        vec![10.0, 25.0, 50.0, 100.0, 200.0, 500.0, 1000.0, 2000.0]
    ).unwrap();

    pub static ref ROUNDS: HistogramVec = register_histogram_vec!(
        "efe_swarm_rounds",
        "Number of fusion rounds before convergence",
        &["verdict"],
        vec![1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0]
    ).unwrap();
```

```
    pub static ref CONVERGENCE_RATIO: Counter = register_counter!(
        "efe_swarm_converged_total", "Converged swarms"
    ).unwrap();

    pub static ref ALIGNMENT_BREACH: Counter = register_counter!(
        "efe_alignment_breach_total", "Alignment projection breaches"
    ).unwrap();

    pub static ref HBM_UTILIZATION: Gauge = register_gauge!(
        "efe_hbm_utilization_ratio", "HBM used / total"
    ).unwrap();
}
```

Dashboard : 1 fichier Grafana fourni (`configs/grafana/efe-overview.json`) avec 12 panneaux.

Effort : 2 jours.

Sous-module 11.3 - Traces OpenTelemetry

Fichier : `src/telemetry/traces.rs`
Span par requete, sous-spans par phase (genesis, round_1..N, dissolution). Attributs : swarm_id, round, template_mix hash. Export via OTLP gRPC vers `otel-collector:4317`.

Effort : 2 jours. **Tests :** spans bien nestes, attributs presents, erreurs remontees en event.
Livrable module 11 : dashboard operationnel, traces lisibles dans Jaeger/Tempo, alerting pret pour 3 regles critiques (latence, breach, erreur).

14. Module 12 - Build, Tests, Deploiement

Infrastructure de developpement. Doit etre solide des le sprint 0, sinon la velocite s'effondre apres.

Effort : 9 jours (repartis sur tout le projet). **Propriete :** SRE.

Sous-module 12.1 - Cargo workspace

Fichier : `Cargo.toml` (racine)

```
[workspace]
resolver = "2"
members = [
    "crates/efe-core",
    "crates/efe-kernels",
    "crates/efe-ffi",
    "crates/efe-server",
]

[workspace.package]
version = "0.1.0"
edition = "2021"
rust-version = "1.80"
authors = ["EFE Team "]

[workspace.dependencies]
tokio = { version = "1.39", features = ["full"] }
tonic = "0.12"
prost = "0.13"
thiserror = "1.0"
serde = { version = "1.0", features = ["derive"] }
cust = "0.3"
uuid = { version = "1.8", features = ["v4", "serde"] }
tracing = "0.1"
opentelemetry = "0.23"
```

Effort : 0.5 jour.

Sous-module 12.2 - pyproject + uv

Fichier : `pyproject.toml`

```
[project]
name = "efe"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = [
  "torch>=2.5,<3.0",
  "triton>=3.0",
  "numpy>=1.26",
  "grpcio>=1.65",
  "protobuf>=5.27",
  "sentence-transformers>=3.0",
  "safetensors>=0.4",
  "opentelemetry-api>=1.25",
  "opentelemetry-sdk>=1.25",
  "prometheus-client>=0.20",
]

[project.optional-dependencies]
dev = ["pytest>=8.0", "pytest-asyncio", "mypy>=1.10", "ruff>=0.5"]

[build-system]
```

```
requires = ["maturin>=1.7"]
build-backend = "maturin"
```

Effort : 0.5 jour.

Sous-module 12.3 - CMake CUDA

Fichier : `CMakeLists.txt`

```
cmake_minimum_required(VERSION 3.28)
project(efe_kernels CUDA CXX)

set(CMAKE_CUDA_STANDARD 17)
set(CMAKE_CUDA_ARCHITECTURES 90)  # H100

add_library(efe_kernels STATIC
    crates/efe-kernels/cuda/spawn_batch.cu
    crates/efe-kernels/cuda/frechet_mean.cu
    crates/efe-kernels/cuda/langevin_sde.cu
    crates/efe-kernels/cuda/projection.cu
)

target_compile_options(efe_kernels PRIVATE
    $<$:
        --use_fast_math
        -Xcompiler=-fPIC
        -O3
        --expt-relaxed-constexpr
    >)

target_include_directories(efe_kernels PUBLIC
    crates/efe-kernels/cuda/include
)

# Tests
enable_testing()
add_subdirectory(tests/cuda)
```

Effort : 1 jour.

Sous-module 12.4 - Docker + CI

Fichier : `docker/Dockerfile.mvp`

```
FROM nvcr.io/nvidia/cuda:12.4.1-cudnn-devel-ubuntu22.04 AS builder

RUN apt-get update && apt-get install -y curl build-essential cmake \
    git python3.12 python3.12-venv python3-pip \
    && curl -L https://sh.rustup.rs | sh -s -- -y --default-toolchain 1.80
ENV PATH="/root/.cargo/bin:${PATH}"

WORKDIR /build
COPY . .

RUN cargo build --release --bin efe-server
RUN cmake -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build -j

FROM nvcr.io/nvidia/cuda:12.4.1-cudnn-runtime-ubuntu22.04
COPY --from=builder /build/target/release/efe-server /usr/local/bin/
COPY --from=builder /build/build/libefe_kernels.so /usr/local/lib/
COPY configs/ /etc/efe/
EXPOSE 50051 9090
ENTRYPOINT ["/usr/local/bin/efe-server", "--config", "/etc/efe/prod.toml"]
```

CI (GitHub Actions) :

```
name: ci
on: [push, pull_request]
jobs:
  rust:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: dtolnay/rust-toolchain@1.80
      - run: cargo fmt --check
      - run: cargo clippy --workspace -- -D warnings
      - run: cargo test --workspace
  python:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: astral-sh/setup-uv@v3
      - run: uv sync --all-extras
      - run: uv run ruff check
      - run: uv run mypy python/efe
      - run: uv run pytest -q
  cuda:
    runs-on: [self-hosted, gpu]
    steps:
      - uses: actions/checkout@v4
      - run: cmake -B build && cmake --build build -j
      - run: ctest --test-dir build
```

Effort : 3 jours (Docker + CI + benchmarks runners).

Livrable module 12 : image Docker publiee, CI verte, CD vers environnement de staging.

15. Plan de sprints

Le MVP est livrable en **7 sprints de 2 semaines** soit 14 semaines calendaires. Les sprints sont pseudo-paralleles (certains modules sont concurrents entre eux).

Sprint	Focus	Livrables
0 (prep)	Setup equipe + infra	Cargo WS, CI de base, envs GPU, kickoff
1 (sem 1-2)	Modules 1 + 2 + 12.1-3	Types, erreurs, memory pool, arenas, stream pool
2 (sem 3-4)	Modules 3 + 4	Genesis engine, ELS instanciable, 1er bout-en-bout minimal
3 (sem 5-6)	Module 5 (fusion)	Swarm attention Triton, 1 round complet, bench < 10 ms
4 (sem 7-8)	Modules 6 + 7 + 8	Consensus + dissolution + 3 templates
5 (sem 9-10)	Modules 9 + 10	Alignment, sandbox, gRPC, client Python
6 (sem 11-12)	Module 11 + integration	Telemetrie, dashboards, tuning perf
7 (sem 13-14)	QA + benchmarks + demo	Benchmarks holdout, documentation, demo publique

Jalons de revue

J1 (fin sprint 2) : *Hello Swarm* - un essaim de 128 agents spawne et l'ELS est populate. Pas encore de convergence.

J2 (fin sprint 4) : *First Convergence* - l'essaim converge sur une tache jouet (addition d'entiers).

J3 (fin sprint 6) : *Alpha demo* - pipeline complet, alignement teste, metriques exposees.

J4 (fin sprint 7) : *MVP GA* - release tagee, benchmarks publies, documentation complete.

Dependances critiques entre sprints

```
S1 ----> S2 : arenas obligatoires pour ELS
S2 ----> S3 : ELS obligatoire pour fusion
S3 ----> S4 : fusion obligatoire pour consensus detection
S4 ----> S5 : dissolution obligatoire avant API end-to-end

En parallele (possibles des S1) :
- S1-S7 : module 12 (build/CI/Docker)
- S2-S5 : module 8 (templates - entrainement teacher en arriere plan)
- S5-S7 : module 11 (observabilite)
```

Risques de sprint

Sprint	Risque principal	Mitigation
S1	Retard sur cust/cuda bindings	Fallback cudarc, 1 jour de buffer
S3	Kernel Triton ne tient pas la perf	Fallback PyTorch flash-attn
S4	Consensus oscille au lieu de converger	Tuning seuils + Langevin sigma
S5	Latence gRPC ajoute 20ms	Garder option in-process API
S6	Dashboards incomplets	Utiliser template pre-fait en attente

16. Criteres d'acceptation du MVP

Le MVP est considere livre si tous les criteres ci-dessous sont verts. Aucun n'est optionnel.

16.1 Criteres fonctionnels

F1. L'API gRPC `Execute` accepte des requetes texte en anglais de <= 2048 tokens et retourne une reponse sans crash.

F2. Le systeme converge sur au moins 95% des 500 requetes du holdout GSM8K-easy.

F3. Les 3 templates sont charges au demarrage depuis safetensors signes.

F4. Un breach d'alignement (requete adversariale) leve une erreur gRPC `FailedPrecondition` avec reason code.

F5. La telemetrie contient un point par round + phase genesis/dissolution.

16.2 Criteres de performance

Metrique	Cible	Mesure
Latence p50	< 150 ms	500 requete
Latence p99	< 500 ms	500 requete
Rounds moyens	< 6	Holdout cor
HBM peak par requete	< 6 Go	NVML mon
Throughput concurrent	> 8 req/s	Batched loa
Cold start	< 8 s	Premier Ex

16.3 Criteres de qualite

Q1. Score qualitatif MVP sur holdout (evalue par LLM judge + human spot-check) : >= 0.85 * teacher_score.

Q2. Taux de breach d'alignement detecte sur `adversarial-v1` : > 99%.

Q3. Couverture tests Rust > 80%, Python > 75%.

Q4. Zero issue critique ou high dans `cargo audit` et `pip-audit`.

16.4 Criteres de livrables et documentation

D1. Image Docker `efe:mvp-1.0` publiee sur registry interne, signee cosign.

D2. Paquet Python `efe==0.1.0` publie sur index interne.

D3. README racine explique : quickstart (5 min), architecture (10 min), contribution (10 min).

D4. Runbook operationnel : deploiement, monitoring, troubleshooting des 10 erreurs les plus frequentes.

D5. ADR (Architecture Decision Records) pour les 5 choix principaux (Rust, CUDA, Triton, gRPC, PyO3).

D6. 3 demos enregistrees (video 2 min chacune) : inference simple, mode debug, breach d'alignement.

17. Risques et mitigations

Analyse systematique des risques prioritaires identifies lors de la phase de design.

17.1 Risques techniques

ID	Description	Severite	Probabilite	Mitigation
T1	Kernel Triton ne compile pas sur certains drivers	Haute	Moyenne	Fallback PyTorch SDPA, CI matrix drivers
T2	Non-convergence sur > 10% des requetes	Critique	Moyenne	Tuning Langevin + NA-CRIT diversification
T3	Mode collapse frequent	Haute	Faible	Garde-fou existant + injection bruit
T4	Fuite memoire HBM apres 1000 requetes	Critique	Faible	Arenas reset-only, test stress 10k req
T5	Latence p99 > 500 ms	Moyenne	Moyenne	Profiling NVIDIA Nsight + CUDA streams
T6	Decodeur texte hallucine massivement	Haute	Moyenne	Tuning distillation + retrieval grounding
T7	Alignment projector trop permissif	Critique	Moyenne	Suite de tests adverses 1000+ cases

17.2 Risques projet

ID	Description	Severite	Probabilite	Mitigation
P1	Indisponibilite GPU H100	Haute	Faible	Accord prealable cluster, A100 fallback
P2	Depart ingenieur CUDA	Critique	Faible	Pair programming systematique, docs
P3	Derive de scope	Moyenne	Haute	Ce document fige + revue hebdomadaire
P4	Dataset BoolQ retire	Faible	Tres faible	Miroir interne
P5	Legal review retarde le release	Moyenne	Moyenne	Legal alignment kickoff S1

17.3 Risques scientifiques

S1. La these est fausse : l'essaim peut ne pas matcher la qualite du teacher en MVP. *Mitigation :* scope reduit aux taches structurees (mots-math, logique booleenne), ou l'essaim a plus de chance. Si echec : pivot vers mode "assist" (aide au LLM) plutot que "remplacer".

S2. Consensus trompeur : l'essaim peut converger sur une reponse fausse avec haute confiance. *Mitigation :* calibration de la confidence, validation croisee avec teacher, garde NA-CRIT renforcee.

S3. Opacite du raisonnement : impossible d'expliquer pourquoi l'essaim a repondu X. *Mitigation :* outils de probing latent (logit lens, trajectory viz) livres en MVP.

18. Annexes

Annexe A - Configuration MVP complete (default.toml)

```
[runtime]
workers = 4
hbm_pool_size_gb = 8
stream_pool_size = 4

[latent]
dim = 2048
precision = "bf16"
manifold = "sphere"

[genesis]
meta_analyzer_model = "all-MiniLM-L6-v2"
meta_head_ckpt = "/var/efe/meta_head_v1.safetensors"
router_mode = "static"
max_agents = 1024
min_agents = 128

[fusion]
strategy = "dense"
max_rounds = 8
attention_heads = 8
head_dim = 256
lora_rank = 16
langevin_eta_schedule = "cosine"
langevin_sigma_init = 0.05
langevin_sigma_final = 0.005

[consensus]
epsilon_variance = 0.001
epsilon_gradient = 0.0001
epsilon_stability = 0.001
stability_window = 3
mode_collapse_guard = true

[alignment]
constitution_path = "/etc/efe/constitution_mvp_v1.sig.sf"
max_delta = 0.15
projection_strength = 0.05

[budget]
max_wall_clock_ms = 500
max_energy_wh = 5.0
max_hbm_bytes_per_request = "6 GiB"

[telemetry]
otel_endpoint = "http://otel-collector:4317"
prom_port = 9090
retain_trajectories_hours = 72
log_level = "info"

[server]
bind = "0.0.0.0:50051"
tls_cert = "/etc/efe/server.crt"
tls_key = "/etc/efe/server.key"
request_timeout_ms = 30000
```

Annexe B - Glossaire

Terme	Definition
ELS	Espace Latent Partage. Manifold R^D sur lequel les nano-agents vivent.
Nano-agent (NA)	Agent ephemere (<50 ms) specialise, position dans l'ELS.
Round	Iteration de fusion (attention + Langevin). MVP: 1-8 rounds.
Consensus	Etat ou la variance intra-essaim et le gradient de potentiel sont sous seuils.
Frechet mean	Moyenne geodesique sur manifold Riemannien. Generalise la moyenne euclidienne.
Dissolution	Phase finale: extraction centroide, decodage, liberation des NA.
Cristallisation	Distillation d'un pattern d'essaim en un NA monolithique (hors MVP).
Ancre	Vecteur prior permanent qui guide la projection d'alignement.
Salience	Poids d'un NA dans la fusion (entre 0 et 1).
Template	Classe de specialisation d'un NA (SEM, LOG, CRIT en MVP).
Mode collapse	Situation ou tous les NA convergent vers le meme point trop tot.
Arena HBM	Region de memoire GPU allouee lineairement, reset a la fin d'une requete.
Langevin SDE	Dynamique bruitee de descente de gradient: dh = -eta grad U dt + sigma dW.
Swarm Attention	Variante d'attention ou Q/K/V sont dependants du template de l'agent.
Breach	Depassement du seuil d'alignement lors de la projection finale.

Annexe C - Metriques de benchmark

Benchmark	Teacher baseline	Cible MVP
GSM8K-easy (500 ex)	0.78	>= 0.66
BoolQ (500 ex)	0.82	>= 0.70
ARC-easy (500 ex)	0.84	>= 0.71
Latence p50 (ms)	240	< 150
Latence p99 (ms)	850	< 500
Energie / req (Wh)	0.8	< 1.5
Rounds moy	n/a	< 6

Annexe D - Points de contact

Role	Contact	Scope
Tech Lead	lead@efe.local	Decisions a
ML Researcher	ml@efe.local	Templates,
CUDA Engineer	gpu@efe.local	Kernels, pe
SRE	sre@efe.local	Deploiemer
Product	po@efe.local	Scope, den
Security	sec@efe.local	Alignment,

--- Fin du document ---

EFE MVP - Sous-modules complets v1.0 - 2026-04-17